



DESARROLLO BIG DATA CON PYTHON

TEMA 5. Machine Learning con
MLlib



Contenidos Tema 5

- / 1. Introducción a MLlib: Machine Learning Distribuido**
- / 2. Conceptos de Machine Learning**
- / 3. Creación de pipelines de ML en PySpark**
- / 4. Entrenamiento, prueba y evaluación de modelos predictivos**

1. MLlib: Machine Learning distribuido con PySpark

- Librería nativa de machine Learning de Spark.
- Scikit-learn, la librería más conocida para machine learning con Python, asume que todos los datos están en una sola máquina.
- **MLlib** proporciona algoritmos y modelos (regresión, Random Forest, K-means, etc.) implementados con una matemática subyacente que permite un entrenamiento en paralelo en un cluster de máquinas.

TRANSFORMAR:

Se recibe un dataframe, y se añade una nueva columna con resultados, p.ej. Predicciones, devolviendo el DataFrame resultante.

- Normalización
- Codificación one-hot
- Inferencia en un modelo

Método equivalente: transform()

ESTIMAR:

Se recibe un DataFrame, se aprende de él (entrenamiento) y se devuelve un modelo que, a su vez, funciona como un *Transformer*.

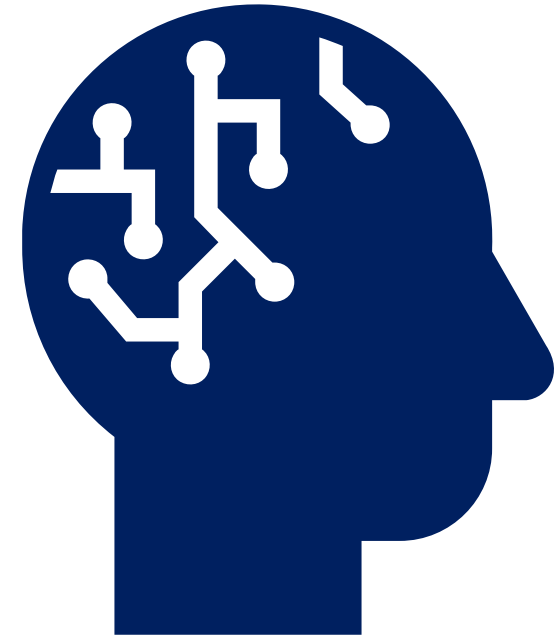
- Ejemplo: RandomForestClassifier

Método equivalente: fit()

INTELIGENCIA ARTIFICIAL

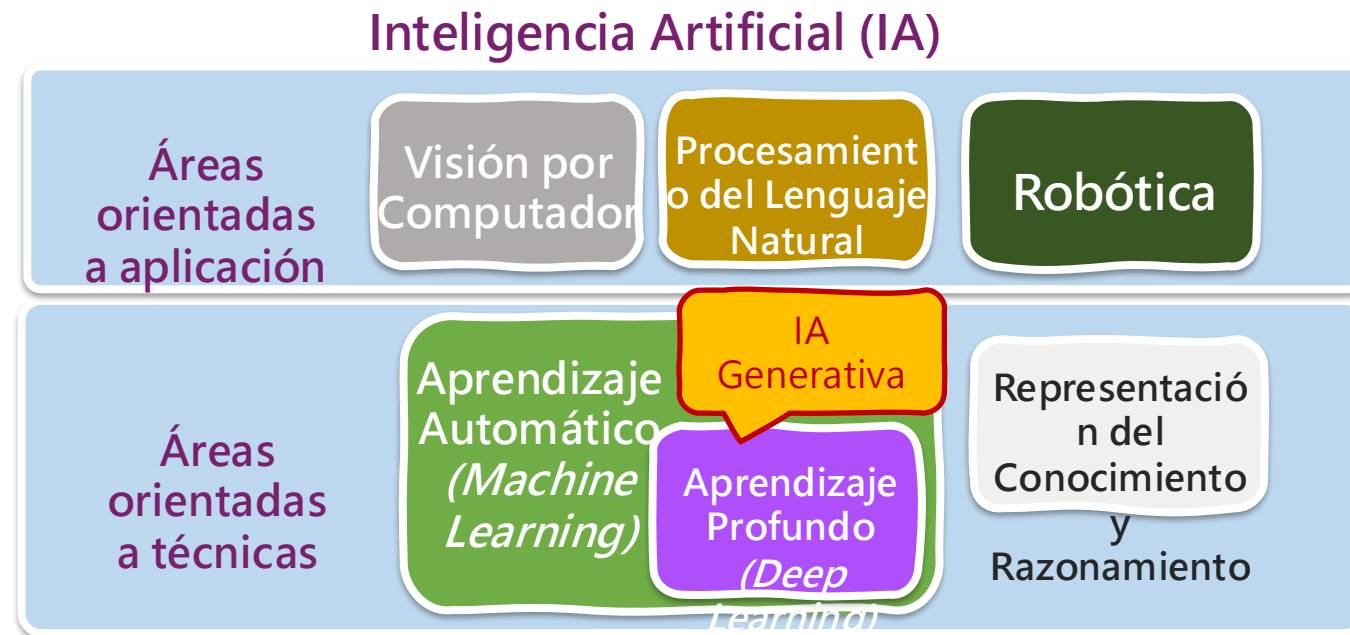
Ciencia e ingeniería centrada en construir máquinas que aprenden a emular ciertos aspectos de la inteligencia humana (o animal) para resolver un problema determinado

- Aprendizaje (a partir de datos)
- Razonamiento: lógico, matemático, visual, ...
- Toma de decisiones
- Comunicación (con humanos, con otros sistemas, ...)



INTELIGENCIA ARTIFICIAL

Ciencia e ingeniería centrada en construir máquinas que aprenden a emular ciertos aspectos de la inteligencia humana (o animal) para resolver un problema determinado



FUNCIONES QUE LA IA PUEDE DESEMPEÑAR

Predicciones e inferencia: función clave de los sistemas de Machine Learning (aprendizaje automático). Aprender a realizar predicciones u otros tipos de inferencia (clasificaciones, estimaciones, pronóstico, ...) en base a datos



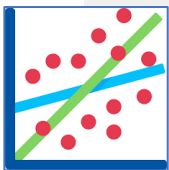
FUNCIONES QUE LA IA PUEDE DESEMPEÑAR

Predicciones e inferencia: tareas de aprendizaje (ML) supervisado

Requisito: disponer de un conjunto de datos de ejemplo cuyo valor de salida ya es conocido □ datos de entrenamiento.

Regresión

Predecir variable continua en función de otras variables relacionadas



Pronóstico de series temporales

Predecir valor futuro continuo a partir de valores pasados con orden temporal



Clasificación

Asignar clase o categoría de nuevos datos o ejemplos a partir de variables



2. Conceptos de Machine Learning

CONCEPTOS BÁSICOS DE MACHINE LEARNING

Aprendizaje Supervisado: tenemos ejemplos de datos *etiquetados* (con clase o variable a predecir conocida) → **datos de entrenamiento**

Cada observación o instancia consta de:

- Uno o varios atributos de entrada/predictores (*features*)
- La salida o variable a predecir: etiqueta (*label*)

Inputs = features **Output
(class/label)**

island	bill length (mm)	bill depth (mm)	flipper length (mm)	body mass (g)	sex	species
Torgersen	39.1	18.7	181	3750	male	Adelie
Torgersen	39.5	17.4	186	3800	female	Adelie
Biscoe	46.3	15.8	215	5050	male	Gentoo
Biscoe	42.9	13.1	215	5000	female	Gentoo
Torgersen	36.7	19.3	193	3450	female	Adelie
Torgersen	39.3	20.6	190	3650	male	Adelie
Torgersen	38.9	17.8	181	3625	female	Adelie

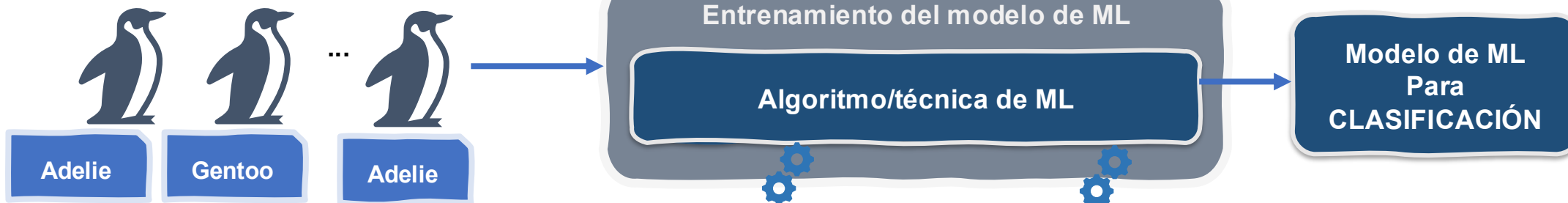
CONCEPTOS BÁSICOS DE MACHINE LEARNING

Mediante **datos de entrenamiento** etiquetados, se **entrena** o construye un **modelo de ML** que aprenderá a realizar una tarea predictiva sobre futuros datos no etiquetados = variable de salida desconocida.

Inputs = features Output
(class/label)

island	bill length (mm)	bill depth (mm)	flipper length (mm)	body mass (g)	sex	species
Torgersen	39.1	18.7	181	3750	male	Adelie
Torgersen	39.5	17.4	186	3800	female	Adelie
Biscoe	46.3	15.8	215	5050	male	Gentoo
Biscoe	42.9	13.1	215	5000	female	Gentoo
Torgersen	36.7	19.3	193	3450	female	Adelie
Torgersen	39.3	20.6	190	3650	male	Adelie
Torgersen	38.9	17.8	181	3625	female	Adelie

Datos de entrenamiento con etiquetas (training data)

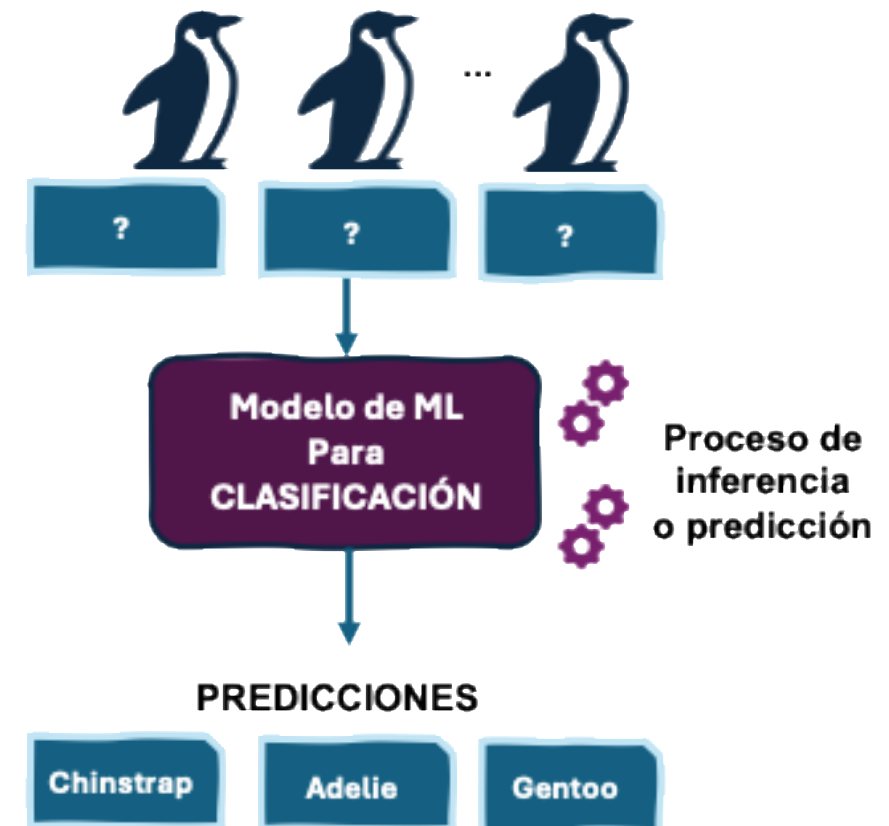


CONCEPTOS BÁSICOS DE MACHINE LEARNING

Aprendizaje Supervisado

Una vez construido el modelo, se utilizar para **realizar inferencia: predicciones sobre datos de prueba** (*test*), evaluar su rendimiento, y posteriormente desplegarlo en entornos donde recibirá datos reales.

Datos de prueba (clase desconocida)

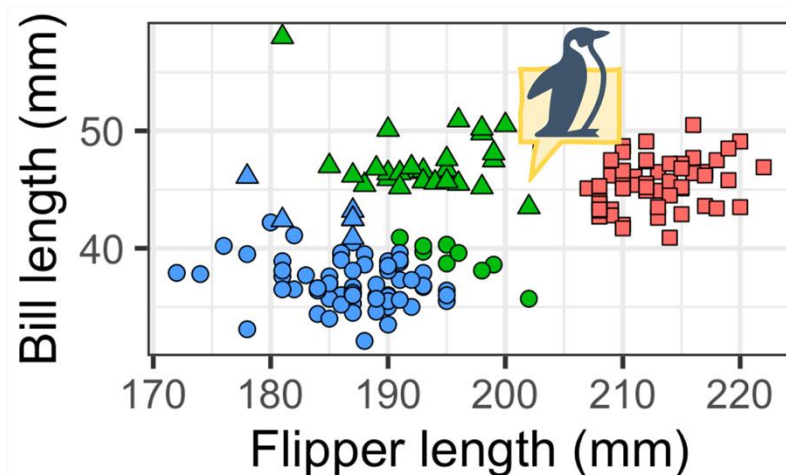


Aprendizaje no supervisado

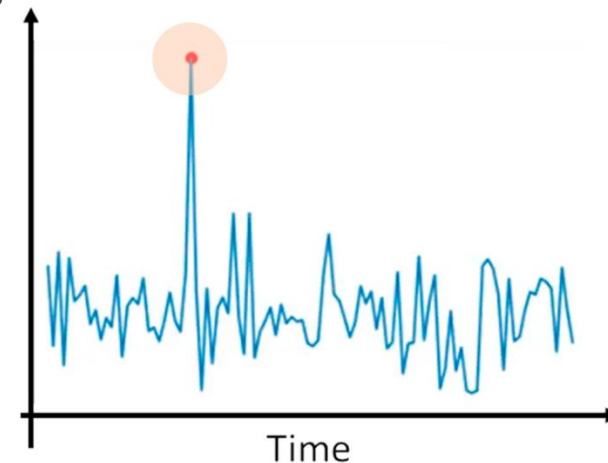
A menudo, los datos históricos reales no están etiquetados.

Los algoritmos de aprendizaje no supervisado extraen conocimiento y patrones de estos datos sin etiquetar.

Clustering: encontrar subgrupos con características similares



Detección de anomalías: identificar observaciones "inusuales", por ejemplo, transacciones bancarias sospechosas

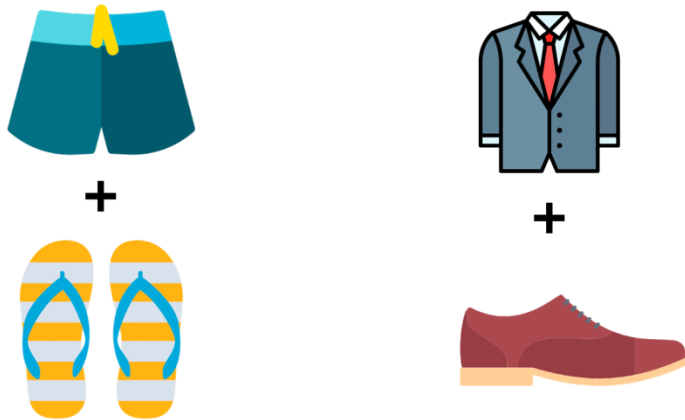


Aprendizaje no supervisado

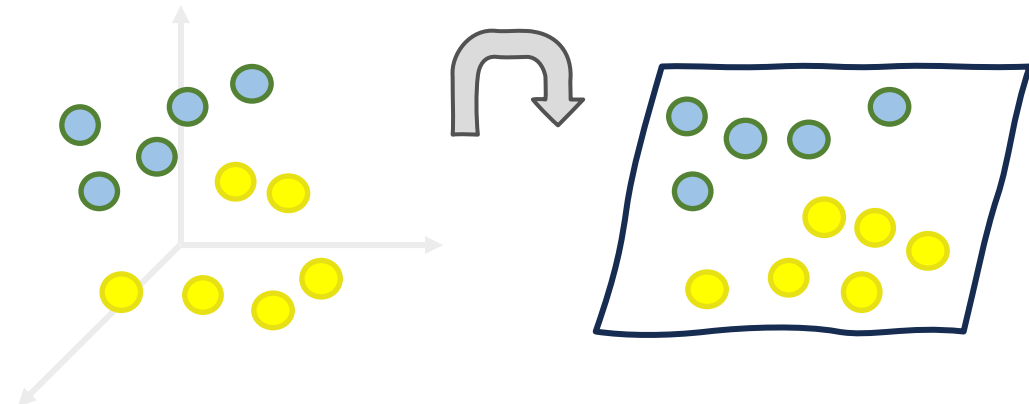
A menudo, los datos históricos reales no están etiquetados.

Los algoritmos de aprendizaje no supervisado extraen conocimiento y patrones de estos datos sin etiquetar.

Descubrimiento de reglas de asociación:
encontrar co-ocurrencias frecuentes en datos de carácter transaccional.



Reducción de la dimensionalidad: reducir número de atributos en los datos, manteniendo sus propiedades clave (útil p.ej. en datos de imágenes)



3. Creación de pipelines para ML

Pipelines de procesamiento de datos con PySpark

- Un Pipeline es un encadenamiento secuencial de Transformers y Estimators.
- Un requisito exclusivo para pipelines en PySpark es utilizar un *VectorAssembler*: todas las variables predictoras (features) encapsuladas en una única columna de tipo Vector.

```
# A. Convertir variable categórica 'type' a numérica
indexer = StringIndexer(inputCol="type", outputCol="typeIndex")

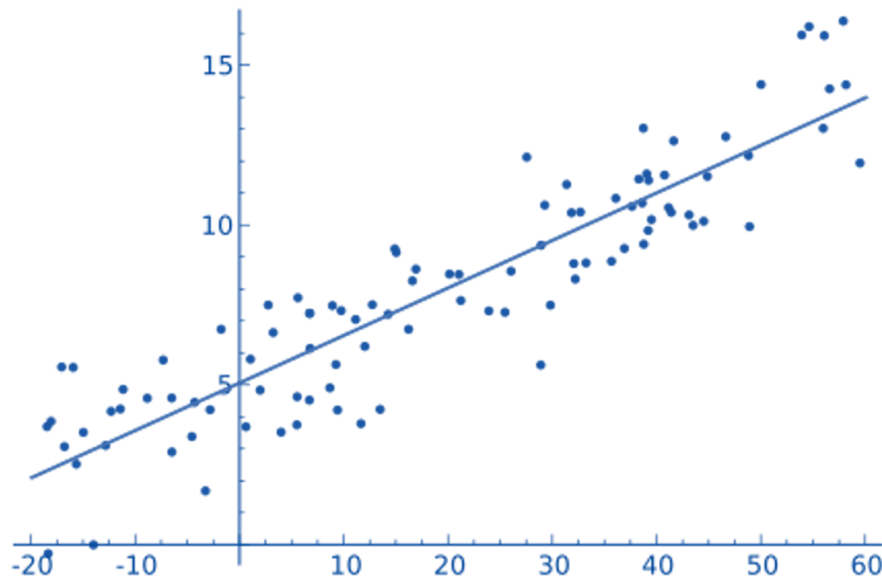
# B. One Hot Encoding para la categoría
encoder = OneHotEncoder(inputCol="typeIndex", outputCol="typeVec")

# C. Ensamblar todas las variables numéricas en un solo vector "features"
columnas_feat = ["amount", "oldbalanceOrg", "newbalanceOrig", "oldbalanceDest",
"typeVec"]
assembler = VectorAssembler(inputCols=columnas_feat, outputCol="features")
[...]
# 4. DEFINICIÓN DEL MODELO
rf = RandomForestClassifier(labelCol="isFraud", featuresCol="features",
numTrees=20)

# 5. CONSTRUCCIÓN DEL PIPELINE
# Esto automatiza el flujo: Indexer -> Encoder -> Assembler -> Modelo
pipeline = Pipeline(stages=[indexer, encoder, assembler, rf])
```

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

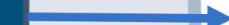
Regresión: aprender a predecir una variable numérica a partir de uno o varios atributos predictores



**House
attributes**



**Regression
Model**



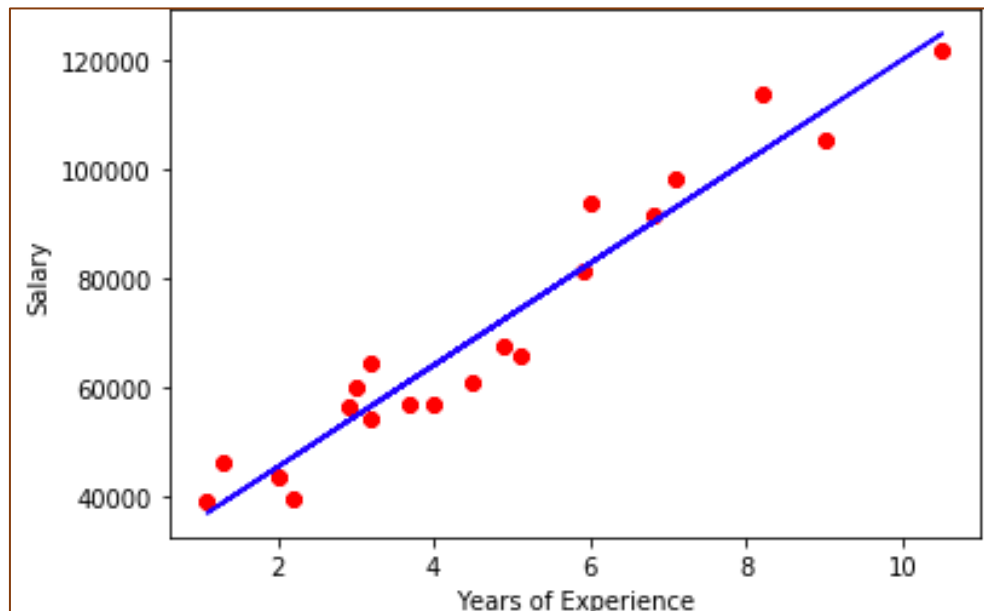
4. Entrenamiento, prueba y evaluación de modelos predictivos

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal simple: Tal vez el modelo de ML supervisado más sencillo de todos

- Estimar variable numérica de salida y a partir de una variable de entrada x , asumiendo una relación lineal entre ambas.

$$\hat{y} = mx + n$$



El modelo aprendido es la **función de una línea recta**, caracterizada por dos parámetros (m , n). Será mejor cuanto más fielmente se ajuste dicha línea a un conjunto de datos de entrenamiento (puntos)

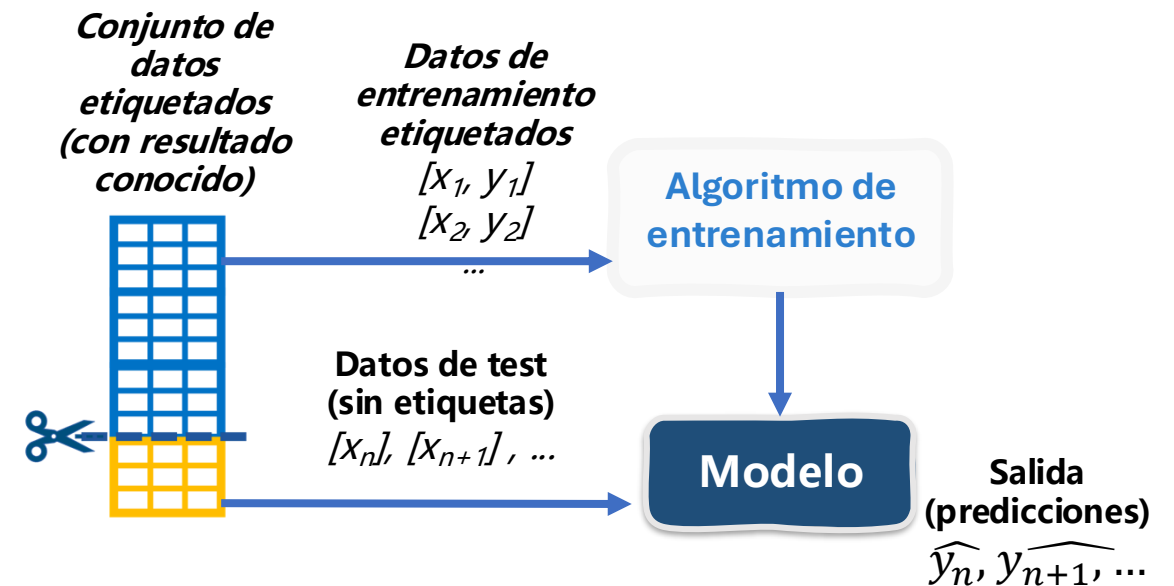
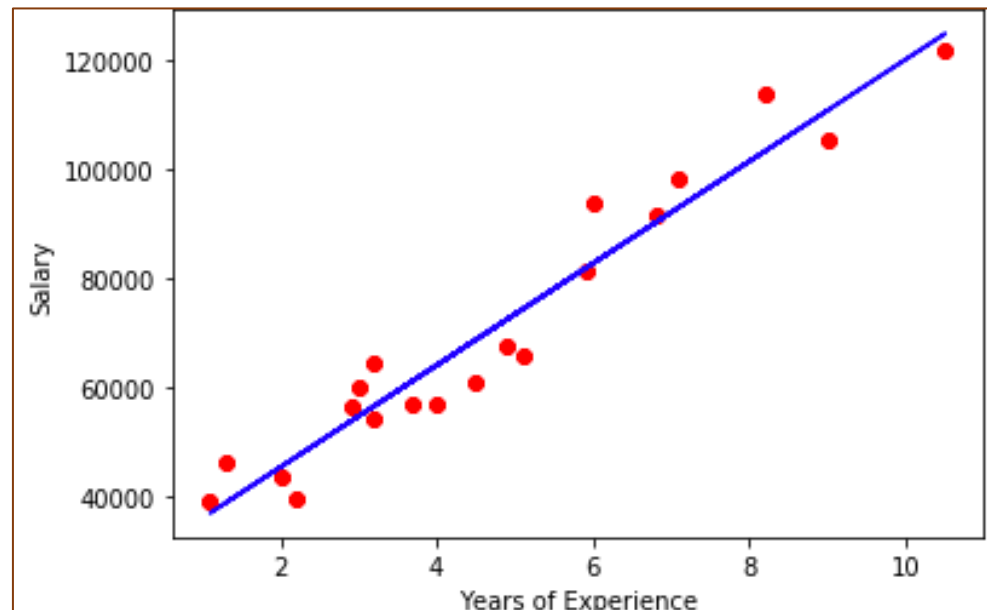
4. Entrenamiento, prueba y evaluación de modelos predictivos

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal simple: Tal vez el modelo de ML supervisado más sencillo de todos

- Estimar variable numérica de salida y a partir de una variable de entrada x , asumiendo una relación lineal entre ambas.

$$\hat{y} = mx + n$$



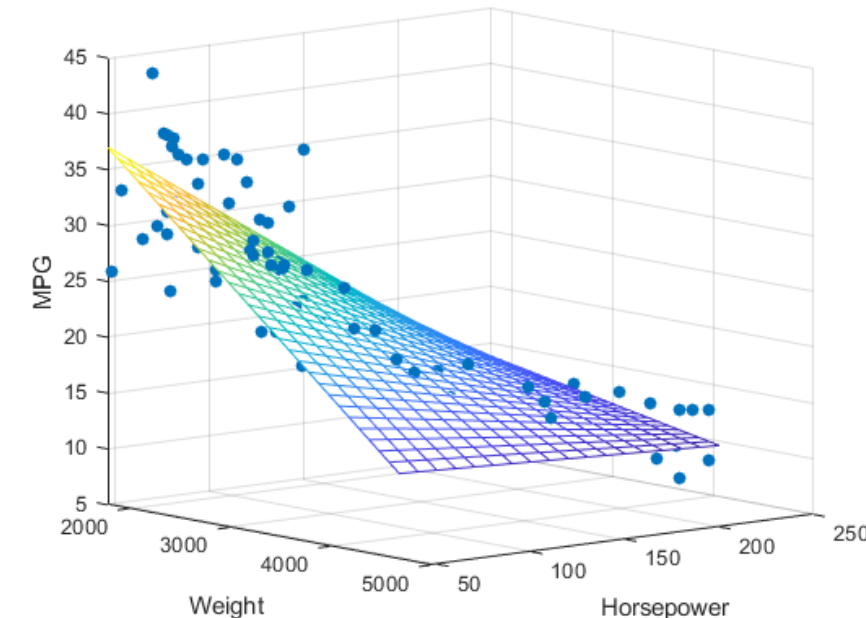
4. Entrenamiento, prueba y evaluación de modelos predictivos

CONSTRUCCIÓN Y ENTRENAMIENTO DE MODELOS DE REGRESIÓN

Regresión lineal múltiple: Estimar variable numérica de salida y , a partir de n variables de entrada $x_1, x_2, \dots, x_n$, asumiendo una relación lineal:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

- Modelo definido por $n+1$ parámetros o pesos θ_j
- El modelo pasa de ser una recta en 2D a ser un plano o, de forma más general, un hiperplano $(n+1)$ dimensional



Tipo de parámetro	Notación	Descripción
Pesos o importancia de los atributos (feature weights)	$\theta_1, \dots, \theta_n$	Un peso θ_j asociado a cada variable predictora. Un mayor peso indica que esa variable es más influyente en las predicciones del modelo.
Término de sesgo (bias term)	θ_0	Necesario para ajustarse a datos de entrenamiento que no pasen por el origen $(0, \dots, 0)$.

